Audio Manager

Contents

- Files
- Setup
- Creating a Sound
- Playing Sounds
- PlaySound Methods
- Volume
- Gotchas

Every sound in the game is a **SoundDataSO** asset holding its clips and settings. The **AudioManager** applies those settings and plays the sound. Designers set sounds up as assets and drag them into components.

This document replaces the old *Audio Manager v2 - Readme*.

See also: Editor Scripts (the SoundDataSO Inspector).

Files

File	Job
`AudioManager.cs`	Persistent singleton with the static `PlaySound` methods.
`SoundDataSO.cs`	The sound asset.
`SoundPlayer.cs`	Drop-in component: plays a sound from a list when told to.
`SoundOnTrigger.cs`	Drop-in component: plays sounds when something enters or leaves a trigger.
`Editor/SoundDataSOEditor.cs`	The custom Inspector.

Sounds play through an **AudioSource on the object making the sound** (sources aren't pooled). The manager has its own 2D source for UI and other non-positional sounds.

Setup

The **AudioManager** lives in the SceneEssentialsBundle. It keeps itself alive between scenes, and any extra copies destroy themselves.

Creating a Sound

Right-click > Create > Audio > Sound Data, give it a clear name (`DoorOpen`, `ImpFootstep`), and fill it in:

Setting	What it does
Clips + **Selection Mode**	One or more clips. Picked **Random**, **Random No Repeat** (good for footsteps) or **Sequential**.
Mixer Group	Routing (SFX, UI, Music).
Volume + **Offset Range**	Base volume plus a random offset each play.
Pitch Range	Random pitch each play. Same min and max = fixed.
Spatial Blend	0 = 2D, 1 = 3D. Distance settings appear above 0.
Loop	Only works with `PlaySound(sound, source)`.
Min Interval	Skips plays that come sooner than this.

Use **Play Preview** at the top to check the clip. It plays at natural volume and pitch, so test the mix in Play Mode.

> **Shared per asset:** the Sequential / No Repeat position and Min Interval are stored **on the asset**, so they're shared by everything using it. Ten enemies sharing a footstep asset step through its clips together, and a Min Interval stops the same sound stacking across all of them. Give objects separate assets if they need their own timing.

Playing Sounds

Situation	Use
UI button, Animation Event	`SoundPlayer`
Something entering or leaving a trigger	`SoundOnTrigger`
An object that already has its own script	`SoundDataSO` fields on that script

SoundPlayer

Holds a **list** of sounds. Call `PlaySound(int index)` from a Button's **On Click**, an **Animation Event**, a UnityEvent or code.

> On enemies, `Enemy` calls `PlaySound(1)` when hit while immune, so keep index 1 for the immune sound.

SoundOnTrigger

Needs a trigger collider. Set **On Enter Sound** and/or **On Exit Sound**, and optionally a **Required Tag** (e.g. `Player`). Works with 2D and 3D colliders.

Sound fields on a script

```
[Header("Audio")]
[SerializeField] private AudioSource audioSource;
[SerializeField] private SoundDataSO openSound;

private void Reset()
{
    audioSource = GetComponent<AudioSource>();
}

public void Open()
{
    AudioManager.PlaySound(openSound, audioSource);
}
```

`Door`, `DoorLock` and `WeaponAudio` all follow this pattern.

PlaySound Methods

Call	Use for
`PlaySound(sound)`	2D sounds through the manager's own source. Fire and forget.
`PlaySound(sound, source)`	One sound at a time on an object, or anything that loops or needs stopping. **Interrupts** whatever that source was playing.
`PlaySound(sound, source, true)`	Sounds that can **overlap** on the same object (rapid hits, footsteps). Can't loop.
`PlaySoundPitched(sound, source, true, pitch)`	One-shot with extra pitch added. Only changes pitch, so the source's own mixer and spatial settings are used.

All of them are safe with a null sound (they log a warning and do nothing) and respect Min Interval.

Volume

- **SoundDataSO Volume** sets a sound's level relative to other sounds.
- **AudioMixer groups** are for player volume sliders.

Don't change asset volumes at runtime for settings. The asset is shared, and Play Mode changes stay on it.

Gotchas

- **No AudioManager = no sound**, with an error on every call.
- **A sound cutting off another on the same object?** Use the one-shot overload or a second AudioSource.
- **Loop only works with `PlaySound(sound, source)`.**